

ComicCraft – AI Comic Story Creator using Gemini Models

Source Documentation – Phase 3: Project Design & Coding

Phase 3 – Project Design & Coding

Objective: This phase converts the ComicCraft requirements into a working web application. The design connects the user interface with a FastAPI backend and the Gemini AI model to generate structured comic stories.

1. System Flow

```
User
↓
Enter Story Prompt
↓
Frontend
↓
FastAPI Backend
↓
Gemini AI Model
↓
Generate Comic Story
↓
Characters + Scenes + Dialogues + Panels
↓
Display Comic
```

The user enters a simple story idea in the web interface. The frontend sends the prompt to the FastAPI backend. The backend sends an instruction to the Gemini model. The generated title, characters, scenes, dialogues, and panel descriptions are returned to the frontend and displayed to the user.

2. Project Structure

```
ComicCraft/
|
├── main.py
├── requirements.txt
├── .env
|
├── templates/
│   └── index.html
|
└── static/
    ├── style.css
    └── script.js
```

3. Backend Coding – main.py

The backend is developed using FastAPI. It receives the story prompt as JSON, creates an instruction for Gemini, sends the request to the model, and returns the generated comic content as JSON.

```
from fastapi import FastAPI
from pydantic import BaseModel
from google import genai
import os

app = FastAPI()

client = genai.Client(
    api_key=os.getenv("GEMINI_API_KEY")
)

class ComicRequest(BaseModel):
    prompt: str

@app.post("/generate")
def generate_comic(request: ComicRequest):

    instruction = f"""
    Create a simple comic story from this idea:

    {request.prompt}

    Give:
    1. Title
    2. Characters
    3. Scenes
    4. Dialogues
    5. Comic panel descriptions
    """

    response = client.models.generate_content(
        model="gemini-2.5-flash",
        contents=instruction
    )

    return {
        "comic": response.text
    }
```

4. Frontend Coding – index.html

The HTML page provides the main ComicCraft interface. It contains a text area for the user's story idea, a button to generate the comic, and an area to display the result.

```
<!DOCTYPE html>
<html>
<head>
    <title>ComicCraft</title>
    <link rel="stylesheet" href="/static/style.css">
</head>

<body>

<h1>ComicCraft</h1>
```

```

<p>AI Comic Story Creator</p>

<textarea id="prompt"
placeholder="Enter your comic story idea..."></textarea>

<button onclick="generateComic()">
  Generate Comic
</button>

<div id="result"></div>

<script src="/static/script.js"></script>

</body>
</html>

```

5. JavaScript – script.js

JavaScript connects the frontend to the FastAPI endpoint. It collects the user's prompt, sends it using a POST request, receives the generated comic, and displays it.

```

async function generateComic() {

  const prompt = document.getElementById("prompt").value;

  const response = await fetch("/generate", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify({
      prompt: prompt
    })
  });

  const data = await response.json();

  document.getElementById("result").innerText =
    data.comic;
}

```

6. CSS – style.css

CSS is used to provide basic layout and styling for the ComicCraft user interface.

```

body {
  font-family: Arial, sans-serif;
  text-align: center;
  padding: 30px;
}

textarea {
  width: 80%;
  height: 150px;
  padding: 10px;
}

button {
  margin: 15px;
}

```

```
padding: 12px 25px;
cursor: pointer;
}

#result {
margin: 20px auto;
padding: 20px;
width: 80%;
text-align: left;
border: 1px solid #ccc;
}
```

7. requirements.txt

The following Python packages are required to install and run the application:

```
fastapi
uvicorn
google-genai
python-dotenv
```

8. Running the Project

Step 1 – Install the required packages:

```
pip install -r requirements.txt
```

Step 2 – Set the Gemini API key in the environment:

```
GEMINI_API_KEY=your_api_key
```

Step 3 – Start the FastAPI development server:

```
uvicorn main:app --reload
```

Step 4 – Open the application in a browser:

```
http://127.0.0.1:8000
```

9. Example Input and Output

Example user input:

```
A young superhero saves a village from a dangerous storm.
```

Expected generated content:

```
Title: The Storm Hero

Characters:
- Arjun - Young superhero
- Maya - Village girl

Panel 1:
Arjun sees a huge storm approaching.
Dialogue:
Arjun: "I must protect the village!"

Panel 2:
Arjun flies toward the village.
```

Panel 3:
Arjun stops the storm and saves everyone.

10. Three Phases Summary

Phase	Name	Main Work
Phase 1	Brainstorming & Ideation	Problem, idea, features, technology
Phase 2	Requirement Analysis	Functional and non-functional requirements
Phase 3	Project Design & Coding	Architecture, UI, FastAPI and Gemini coding

11. Conclusion

Phase 3 provides the technical design and initial implementation of ComicCraft. The system uses HTML, CSS, and JavaScript for the frontend, FastAPI for the backend, and Gemini for AI-based comic story generation. This structure provides a foundation for further development, including improved comic-panel rendering, image generation, database support, authentication, and deployment.